hw16

With the election coming up, we'll be focusing on analyzing polling data for US Senate races. The data is taken from the New York Times - the home page for their poll tracker can be found here:

https://www.nytimes.com/interactive/polls/latest-polls.html

Here's the link to the raw data, though you don't need to download it:

https://www.nytimes.com/newsgraphics/polls/senate.csv

I've already done some processing of this data. In particular, I've filtered it to only include polls for the general election (all primary polls have been removed) and to only include polls with the final candidates (some polls that were done before the primary used hypothetical match-ups before the candidates were set, so only polls that use the final candidates were retained). The data is contained in the file "senate_processed.csv".

Note: The goal of this assignment is to get comfortable with `dplyr`, so use `dplyr` for all of your data processing.

1 Convert to dates

For this assignment we'll need to work with dates, which we haven't covered yet. R has a special class for dates, so we'll need to convert the dates in the data frame to be of class `Date`. To make things a bit easier, we'll use functions from the `lubridate` package (you'll need to install this package if it's not already installed).

We'll need to convert the two columns that contains dates (`start_date` and `end_date`) to be of class `Date` - to do this, use the `ymd()` function (you'll need to load the `lubridate` package first). Here's a small example of how you could do this for a single column:

```r
library(lubridate)
df <- data.frame(date = c("2021-06-07", "2024-08-30"))
df |>
  mutate(date = ymd(date))
```

Convert the `start_date` and `end_date` columns to be of class `Date`.

2 Summarize the recent Iowa results

2.1 *Average for October*

Using the polls that ended on or after 2026-10-01, find the average value for `pct` for only the Republican (REP) and Democratic candidates (DEM) in the Iowa senate race. The result should be a data frame with two rows - one for the Democrat and one for the Republican.

Note: to get dates more recent than a certain date, you can write something like `start_date >= "2026-10-01"` in your call to `filter()` - your string will be interpreted as a date. Just make sure it's in year-month-day format, as shown.

2.2 *Get the weeks*

We'd like to get the average poll numbers for each week and month. To do this, we first need to identify the week/month each poll occurred in (for example, we need a way to indicate that "2026-10-20" and "2026-10-23" are part of the same week). One way to do that is to "round" each date to the first date in that week/month. `lubridate` provides a convenient function that does this for us: `floor_date()`. Read the help file (or do some web searching) to figure out how to use this function, and then create two new columns using `floor_date` - `week` that contains the week and another (`month`) that contains the month. Use the `end_date` column to calculate these columns.

3 Aggregate by week

Using the new columns you just created, create a data frame containing the polling data aggregated by month and another data frame with the data aggregated by week. The resulting data frames should show each candidate's average approval rating in that month/week across all polls conducted in that time period.

3.1 *Plot the swing states*

As of Oct. 26, 2026, the Cook Political Report lists seven Senate races as being toss-ups:

- Alaska (AK)
- Iowa (IA)
- Kansas (KS)
- Maine (ME)
- Michigan (MI)
- Ohio (OH)
- Texas (TX)

Using the aggregated data from the previous create a faceted plot (one pane per state) that plots the aggregated results over time. Only include the results for Democrats (DEM) and Republicans (REP). Color the points by political party.

Create one plot that shows the weekly average and another plot that shos the monthly averages.

Note - you may notice that the default color palette selected by `ggplot()` is perhaps not what you would normally expect in this context - do some online searching to see how to set the color palette and set the colors to be more conventional.

3.2 *Compute the Republican-Democrat gap*

Instead of looking at the raw values, let's take a look at the gap between the Democrat and Republican candidates. We'll use the monthly aggregated data that we just calculated.

3.2.1 *Reshape the data*

This data is in "long" format - each poll has multiple rows. Let's widen the data so that each state and month has a single row and there are separate columns for the parties. The resulting data frame should have one column per political party contained in the `party` column. Before widening, subset the data to only include rows for candidates that are Republican (REP), Democrat (DEM), or Independent (IND). For this analysis we'll ignore the candidate's name

and focus only on their party. Also add a column that contains the difference between the Republican and Democrat candidates (Republican - Democrat).

The resulting data frame should have the following 6 columns:

```
"state"   "month"  "DEM"   "REP"   "IND"  "REP_DEM_dif"
```

3.2.2 *Create a plot*

Create plot that shows the difference between the Republican and Democratic candidates for the seven toss-up states.

3.3 *Rank states by difference*

Using only the last month of data, add a column that rates the state as "Solid D", "Likely D", Leans D", "Toss-up", "Leans R", "Likely R", "Solid R" base on the value of `REP_DEM_dif` - choose cutoff values that seem to make sense (do you remember what function we've used in the past to categorize numeric data?). Display the data frame sorted by `REP_DEM_dif` and only show the columns containing the state, the Republican-Democrat difference, and the rating you assigned to it.

Note: you'll notice that South Dakota (SD) and Idaho (ID) have `NA's` for the difference because there is no Democrat running. Remove these two states

3.3.1 *Bar plot of the results*

Using the data frame from the previous question, create a bar plot showing the results (note - you'll want to be using `geom_col()`). The x axis should have the state name and y axis should have the value of `DEM_REP_dif` (so some bars will be negative). Do some searching to find out how to order the bars so that the bars go from low to high. Also, color the bars (you'll want use `fill`) by the rating that you assigned in the previous question. If you want a set of 7 colors that might fit for the bars, here's some code to generate a set of 7 colors you could use (though you can use a different set if you want). You'll want to make use of `scale_fill_discrete()` to set the color (check the documentation or do some searching to find out how to use it).

```
cols <- colorRampPalette(c("blue", "#d59bfa", "red"))(7)
```